Which Optimisations Pay Off? Loop Reordering, Unrolling and Cache Blocking for Dense Matrix Multiplication, and When CSR Sparse Multiplication Wins

Big Data – Individual Assignment 2: Optimised Matrix Multiplication Approaches and Sparse Matrices

Aimar Daniel Alejandro Santana
Grado en Ciencia e Ingeniería de Datos, Universidad de Las Palmas de Gran Canaria
Repository: https://github.com/aimarsantana/BigData-Assignment02-MatMul
October 2026

Abstract. We study two questions on one machine (AMD Ryzen 7 7730U, Windows 11, single thread, C, binary64): (A) how much three sequential optimisations—loop reordering (ikj), k-loop unrolling by 4, and cache blocking—speed up the basic triple loop (ijk) of Assignment 1, and (B) for which tested sparsity levels and input structures a CSR (Gustavson) sparse product beats dense multiplication on the same logical matrices. All methods read the same input files, give bit-identical results, and are timed on the kernel only (median and IQR of 3–10 repetitions, warm-up excluded). Loop reordering alone makes the dense product 16× faster at n = 1024; adding a 4-fold unrolling reaches 24× (n = 1024) and 21× (n = 2048). Blocking helps ikj only once the matrix B no longer fits in the 16 MB L3 cache (1.3–1.4× for n ≥ 1536), but did not beat the unrolled kernel in our tests. At n = 1024 the CSR product is slower than the best dense method on fully dense inputs (0.29×), roughly on par at 50 % zeros (1.07×), and 10–250× faster at 90–99.5 % zeros. Dense→CSR conversion costs about 3 ms at n = 1024 whatever the sparsity, so it limits the advantage at very high sparsity (11,700× for the kernel alone, 85× with conversion at 99.9 %). With the same density, banded inputs (no fill-in) are 15× faster than uniformly random ones. The conclusions hold for the tested code, inputs and hardware only.

1. Introduction and objective

Assignment 1 measured a basic triple-loop multiplication in three languages and found that, even in C, the cost per inner iteration grew with n because the column-wise access to B leaves the caches. This second study takes the C baseline and asks what the simplest source-level optimisations buy, and when exploiting zeros pays off. Dense matrices are n×n; n is the side length.

Research questions. (A) How do loop reordering, loop unrolling and cache blocking change the runtime and memory of a sequential dense binary64 multiplication relative to the baseline, as n grows? **(B)** For which declared sparsity levels (percentage of zero entries) and sparsity structures does a CSR sparse product beat dense multiplication on matched logical inputs, once conversion, indexing and the density of the output are taken into account?

Predictions (formulated from the background in Section 2 before the main runs; the short debugging runs at n ≤ 2048 that preceded them showed that blocking gave no gain over ikj at n = 2048 with the first tile sizes tried):

1. **H1.** ikj is at least 5× faster than ijk for n ≥ 1024, because the inner loop becomes unit-stride in B and C.
2. **H2.** Unrolling the k loop by 4 adds a modest further gain (1.2–2×), because C[i][*] is loaded and stored once per four multiply-adds.
3. **H3.** Blocking helps (relative to ikj) only when B no longer fits in the last-level cache (n ≳ 1500 here); the best tile side is a few tens of elements.
4. **H4.** With uniformly random zeros, the CSR product overtakes the best dense method at roughly 90 % zeros; counting conversion moves this point to higher sparsity.
5. **H5.** Sparsity alone does not determine performance: the product of random sparse matrices fills in (so output cost stays high), whereas banded matrices of the same density do not.

2. Background

Dense product and loop order. C=AB needs n^3 multiplications and n^3 additions ($2n^3$ flops) [1]. All six loop orders perform the same arithmetic but traverse memory differently. In the baseline ijk order the inner loop walks down a column of B with stride n, so for large n each access touches a new cache line and a new page [5, 8]. In ikj order the inner loop reads a row of B and updates a row of C with unit stride, which suits hardware prefetching and cache lines. **Loop unrolling** replicates the loop body to reduce loop overhead and, here, memory operations on C. **Cache blocking** (tiling) restructures the loops to work on sub-matrices that fit in a cache level so that every loaded element is reused many times [2]; high-performance libraries combine it with register tiling and SIMD [6]. We did not implement Strassen's sub-cubic algorithm [7]: its recursion overhead and numerical behaviour make it a separate study.

Sparse matrices. A matrix with mostly zero entries can be stored in CSR (compressed sparse row) format: arrays `val` (non-zero values), `col` (column indices) and `rowptr` (start of each row) [4, 11]. Gustavson's algorithm [3] multiplies row by row: for every non-zero A[i][k] the scaled row k of B is merged into a dense accumulator indexed by column, and the touched columns are then gathered (and sorted) into row i of C. Its work is proportional to the number of non-zero products, not to n^3, but each product costs an indirect access. If A and B have density d with independent random positions, an entry of C is structurally non-zero with probability $1-(1-d^2)^n$, so the product of two sparse random matrices quickly becomes dense ("fill-in"); banded matrices of half-bandwidth w give a product of half-bandwidth 2w.

3. Methodology

3.1 Implementations

Table 1. Methods (all in `src/c/matmul2.c`, C11, binary64, one thread; own code unless stated).

Method	Change relative to the previous row / expected effect
ijk	Baseline reused from Assignment 1 (`matmul.c`): triple loop, scalar accumulator, row-pointer storage `double**`. Column-wise (stride-n) access to B.
ikj	Loop reordering: C[i][*] += A[i][k]·B[k][*]; unit stride in B and C; C is zeroed first (inside the timer).
unroll4	ikj with the k loop unrolled by 4 (four rows of B folded into C[i][*] per pass, summed in increasing k): fewer loads/stores of C per flop.
blocked	ikj kernel on tiles of side b (loops ii, kk, jj; k blocks in increasing order). b = 32 fixed from the sweep of Section 4.2.
csr	Sparse×sparse product on CSR (int32 indices, binary64 values), Gustavson with dense accumulator, sorted output columns; own code, no library.
convert	Dense→CSR conversion of A and B (own code), timed separately from the product.

Every dense kernel adds the products of C[i][j] in increasing k starting from +0.0, with no FMA contraction (generic x86-64 target, no `-ffast-math`); so the results are expected to be *bit-identical*, which we verify. The compiler (gcc 16.1.0, `-O2`) emitted only scalar SSE2 instructions (`mulsd/addsd`) in the four dense kernels (checked with `objdump -d`): no SIMD vectorisation occurs, so differences between methods are due to memory access and instruction count, not to vector width. Parallel and vectorised variants are out of scope (Assignment 3). Baseline and optimised methods use the same storage, numeric type, input matrices and timing boundary.

3.2 Inputs

Dense (Part A). The matrices of Assignment 1: NumPy PCG64, seeds 20261012 (A) and 20261013 (B), U[−1,1), binary64, little-endian row-major files, n ∈ {64,…,4096}; the SHA-256 of all 18 regenerated files that also exist in Assignment 1 matched its manifest. **Sparse (Part B).** Sparsity is the percentage of zero entries (density = 100 − sparsity). *Uniform*: each row of A and B has exactly round(n(1−sparsity)) non-zeros at uniformly random distinct columns (position seeds 20261026/27, value seeds 20261028/29, values U[−1,1)). *Banded*: entry (i,j) is non-zero iff |i−j| ≤ w. Each case is written in two representations of the same logical matrix (dense file for the dense methods, CSR file for the sparse method; checked equal). The sparsity-0 % case uses the dense matrices of Part A. Experiments: **D1** dense size sweep (n = 64–4096); **D2** tile-size sweep;

S1 sparsity sweep (n = 1024; 0, 50, 80, 90, 95, 98, 99, 99.5, 99.9 % zeros); **S2**/**S3** size sweeps at 99 % and 99.9 % zeros; **S4** banded widths w ∈ {0,2,5,13,51} (n = 1024); **S5** banded size sweep (w = 5). Dense methods were run on the matched dense file of each sparse case (all four methods in S1, S2, S4; blocked only in S3, S5; no dense input was generated above n = 4096).

3.3 Environment

Table 2. Measurement environment (`results/environment.txt`).

CPU	AMD Ryzen 7 7730U (laptop), 8 physical cores / 16 logical processors, L2 4 MB total (8×512 KB), L3 16 MB, max clock reported by Windows 2.0 GHz (turbo not controlled)
Memory / OS	13.84 GB RAM (2×8 GB DDR4-3200); Windows 11 Home build 26200; AC power, "Balanced" power plan
Toolchain	gcc 16.1.0 (WinLibs MinGW-w64 UCRT), `-O2 -std=c11 -Wall -Wextra -lpsapi`; Python 3.14.5, NumPy 2.5.4, pandas 3.0.6, matplotlib 3.11.2 (inputs, validation, analysis only)
Threads	1. Process pinned to one logical processor (#2), high priority; no numerical library is called in any timed region.

3.4 Correctness checks

`tests/validate.py` ran 95 checks (**95 pass, 0 fail**) before benchmarking: a hand-computed 3×3 product, identity and zero matrices, sizes not divisible by the unroll factor or the tile (n = 1, 2, 5, 33, 100, 127, 130 with tiles 1, 7, 32, 64, 500), sparse matrices with forced empty rows/columns and extreme densities (0 and 100 %), the benchmark inputs, and a check that the CSR structure of the output (sorted unique columns, nnz equal to the structurally reachable positions) is valid. The reference is NumPy (OpenBLAS), used only for checking. The tolerance is $|\Delta C| \le 10^{-9} + 10^{-9}|C|$: the rounding error of a length-n dot product with entries in $[-1,1]$ is bounded by $n \cdot u \cdot \sum|a_{ik}b_{kj}| \le n^2u = 1.2\times10^{-10}$ for n = 1024 ($u = 2^{-53}$), so 10^{-9} is above any legitimate rounding error and far below logic errors (largest error on the benchmark inputs 1.4×10^{-13}). In addition, every optimised kernel must equal the baseline *bit for bit* (78 comparisons, all equal), and the result checksum was compared across all methods for each of the 46 benchmark configurations (disagreements: 0).

3.5 Measurement protocol

- **Timer boundary (kernel only).** *Dense:* the multiplication, including zeroing of C for ikj/unroll4/blocked; C is allocated outside the timer. *CSR:* the whole product including allocation and growth of the output (its size is data dependent), which makes the sparse boundary slightly wider than the dense one. *Conversion:* both dense→CSR conversions including allocation. Inputs are loaded and results are checksummed outside the timer. The two sparse timings are reported separately and as their sum.
- **Timer and repetitions.** `QueryPerformanceCounter` (monotonic). One fresh process per configuration. Warm-up: 2 runs (n ≤ 512) or 1 run, excluded from statistics but kept in the raw data. Measured repetitions: 10 (n ≤ 512), 5 (n ≤ 2048), 3 (n > 2048; fewer because a run takes up to 40 s), and 10 for the sparse kernels. Short kernels are batched automatically so that a sample lasts ≈20 ms (the time per multiplication is reported; the batch is stored in every row). The result is consumed through a checksum.
- **Statistics.** Median and interquartile range (IQR) of the measured samples; all raw samples are in `results/raw_samples.csv`. No sample was removed.
- **Budgets.** 120 s per sample and 4 GB peak working set. A series that exceeds the budget is not attempted at larger sizes; timeouts are reported as results, with no invented time. The largest *tested feasible* size and the largest *attempted* size are reported separately.
- **Memory metric.** Peak working set (`PeakWorkingSetSize`, the Windows analogue of peak resident set) of the benchmark process, read by the process itself via `GetProcessMemoryInfo` [10]; the peak commit charge is also stored. It includes the runtime, the loaded inputs and all temporaries, not only the matrices; it is compared with the analytic size of the arrays ($8n^2$ bytes per dense matrix; $12 \cdot nnz + 4(n+1)$ bytes per CSR matrix).

4. Results

4.1 Part A: dense optimisation

Table 3. Dense methods: median time per multiplication (IQR in parentheses; 10 repetitions for n ≤ 512, 5 for n ≤ 2048, 3 above). Entries without a time are runs that hit the declared budget.

n	ijk (baseline)	ikj (loop reorder)	ikj + unroll x4	blocked (tiled ikj)
64	227.7 µs (8.2 µs)	118.9 µs (13.5 µs)	89.4 µs (3.3 µs)	139.4 µs (30.2 µs)
128	4.08 ms (92.6 µs)	1.06 ms (49.0 µs)	735.7 µs (13.2 µs)	1.03 ms (66.7 µs)
256	57.79 ms (535.6 µs)	8.88 ms (1.11 ms)	5.88 ms (332.7 µs)	8.63 ms (209.7 µs)
384	108.05 ms (1.27 ms)	27.65 ms (680.4 µs)	18.50 ms (743.9 µs)	27.10 ms (1.02 ms)
512	791.20 ms (2.41 ms)	48.48 ms (1.41 ms)	33.65 ms (1.39 ms)	58.90 ms (5.36 ms)
768	1.41 s (2.38 ms)	165.77 ms (2.89 ms)	110.21 ms (5.67 ms)	180.41 ms (8.25 ms)
1024	6.57 s (22.84 ms)	404.09 ms (22.47 ms)	275.26 ms (5.44 ms)	455.74 ms (8.70 ms)
1536	21.22 s (17.75 ms)	2.00 s (91.33 ms)	1.11 s (3.20 ms)	1.53 s (33.93 ms)
2048	58.31 s (117.79 ms)	4.93 s (16.16 ms)	2.72 s (24.34 ms)	3.54 s (48.15 ms)
3072	timeout	17.05 s (676.34 ms)	9.63 s (89.43 ms)	11.95 s (95.04 ms)
4096	not attempted	40.67 s (1.51 s)	23.41 s (426.78 ms)	28.48 s (159.55 ms)

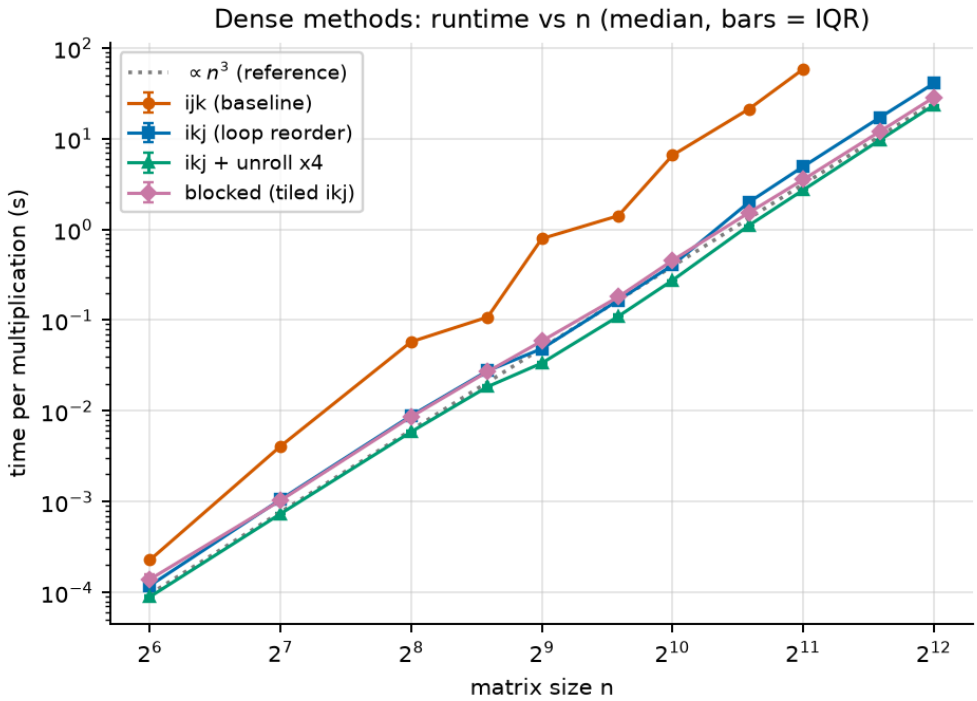


Figure 1. Time per multiplication of the four dense methods (log–log, median and IQR; the dotted line is proportional to n^3). The ijk series stops at n = 2048: n = 3072 exceeded the 120 s budget (TIMEOUT), n = 4096 was not attempted.

Table 4. Speed-up over the ijk baseline (ratio of medians) wherever the baseline completed within the budget.

n	ijk time	ikj	ikj + unroll x4	blocked
64	227.7 µs	1.92×	2.55×	1.63×
128	4.08 ms	3.87×	5.55×	3.96×
256	57.79 ms	6.51×	9.82×	6.70×
384	108.05 ms	3.91×	5.84×	3.99×
512	791.20 ms	16.32×	23.51×	13.43×
768	1.41 s	8.52×	12.82×	7.83×
1024	6.57 s	16.25×	23.85×	14.41×
1536	21.22 s	10.63×	19.16×	13.86×
2048	58.31 s	11.84×	21.45×	16.46×

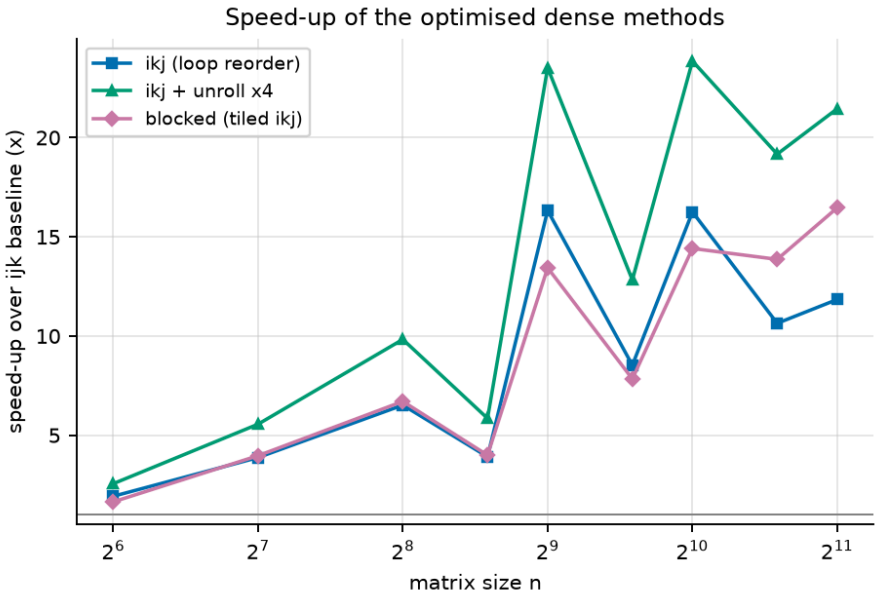


Figure 2. Speed-up of the optimised methods over the ijk baseline (ratio of medians).

The median relative IQR over all dense configurations is 2.3 % (maximum 22 %, at the smallest sizes), so the differences discussed below (≥ 30 %) exceed the run-to-run spread inside a session. The fitted log–log slopes of time versus n ($n \geq 256$) are 3.43 (ijk, up to 2048), 3.11 (ikj), 3.03 (unroll4) and 2.93 (blocked, up to 4096), against 3 for the pure operation count. The baseline is not monotone in n: the sizes that are multiples of 512 (512, 1024, 1536, 2048) cost 5.9–6.8 ns per n^3, about twice the 1.9 and 3.1 ns of n = 384 and 768, consistent with a large power-of-two-related column stride in B (Assignment 1 reported a similar effect).

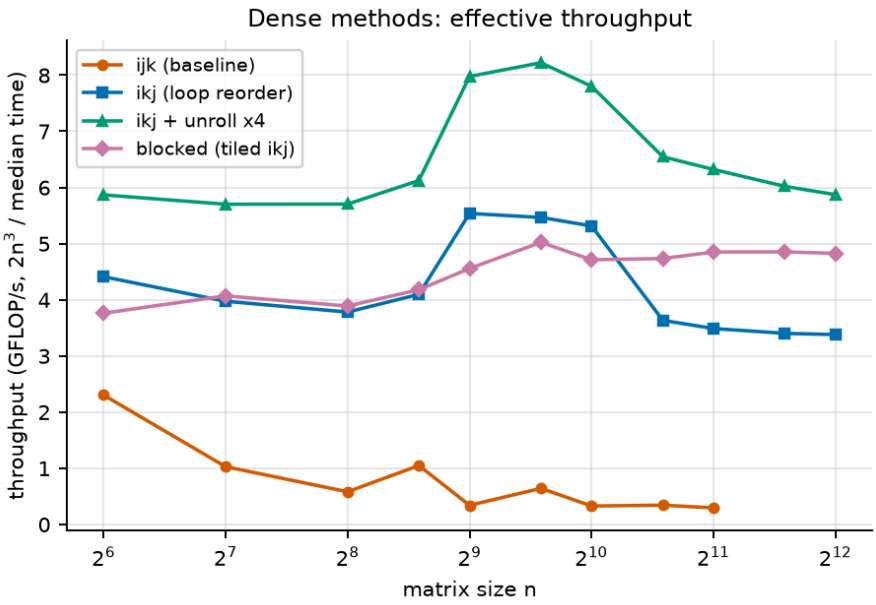


Figure 3. Effective throughput $2n^3$/median time. The ijk baseline stays near 0.3 GFLOP/s for n ≥ 1024; the optimised methods reach 3–8 GFLOP/s.

4.2 Tile-size sweep (blocked method)

Table 5. Block-size sweep of the blocked kernel (median time per multiplication, 5 repetitions; 3 for n = 4096).

n	b=8	b=16	b=32	b=64	b=128	b=256	b=512	best b
1024	524.65 ms	411.50 ms	353.60 ms	333.33 ms	388.29 ms	367.95 ms	332.30 ms	512
2048	3.98 s	3.61 s	3.14 s	3.30 s	3.79 s	3.84 s	3.96 s	32
4096	39.91 s	30.34 s	23.29 s	29.35 s	34.15 s	31.30 s	28.20 s	32

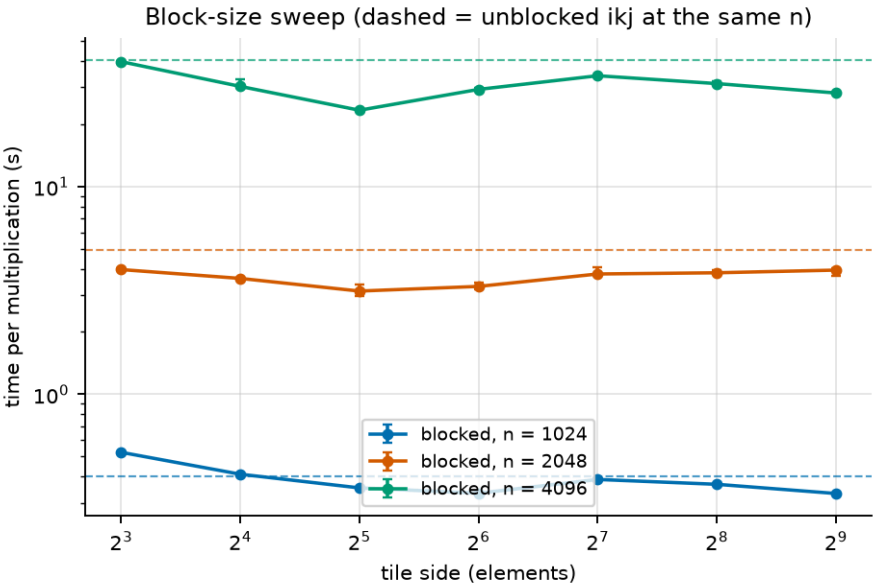


Figure 4. Tile-size sweep of the blocked kernel (dashed lines: unblocked ikj in the main experiment D1 at the same n).

Tiles of 32 (best at n = 2048 and 4096) to 64 (best at 1024 within 0.5 % of 512) elements are best, and tiles of 8–16 are 25–70 % slower. The main experiments use b = 32, chosen from this sweep, which was run on the same machine and sizes as the evaluation, so the blocked results are mildly tuned in its favour. The sweep was run in an earlier session than D1: the same configuration (n = 1024, b = 32) took 354 ms there and 456 ms in D1, a 29 % drift between sessions on this laptop (thermal state, turbo clock, background load),

which bounds how finely cross-session numbers can be compared. Comparisons inside D1 (same session) are more reliable.

4.3 Part B: sparse matrices

Table 6. Sparsity sweep at n = 1024 (uniform random positions, 10 repetitions for the sparse kernels). A ratio above 1 means the sparse kernel is faster. nnz(C) is the structural non-zero count of the product.

zeros (%)	nnz(A)	nnz(C)	density of C	CSR product	dense→CSR (A+B)	dense ijk	best dense	ijk / CSR	best dense / CSR	best dense / (CSR + conv.)
0	1,048,576	1,048,576	100.0%	944.31 ms	7.01 ms	6.34 s	269.51 ms (unroll4)	6.72×	0.29×	0.28×
50	524,288	1,048,576	100.0%	250.20 ms	12.78 ms	6.14 s	268.15 ms (unroll4)	24.55×	1.07×	1.02×
80	209,920	1,048,576	100.0%	57.83 ms	6.73 ms	6.30 s	274.74 ms (unroll4)	108.85×	4.75×	4.26×
90	104,448	1,048,563	100.0%	26.54 ms	4.83 ms	6.32 s	274.89 ms (unroll4)	238.02×	10.36×	8.76×
95	52,224	971,593	92.7%	18.49 ms	3.85 ms	6.57 s	270.00 ms (unroll4)	355.44×	14.60×	12.08×
98	20,480	341,758	32.6%	6.82 ms	3.05 ms	6.39 s	273.30 ms (unroll4)	937.41×	40.10×	27.69×
99	10,240	97,951	9.3%	5.68 ms	2.91 ms	6.47 s	262.17 ms (unroll4)	1139.75×	46.16×	30.53×
99.5	5,120	25,354	2.4%	1.08 ms	2.90 ms	6.74 s	267.85 ms (unroll4)	6232.00×	247.52×	67.24×
99.9	1,024	1,024	0.1%	23.1 µs	3.18 ms	6.87 s	271.03 ms (unroll4)	297202.34×	11730.55×	84.60×

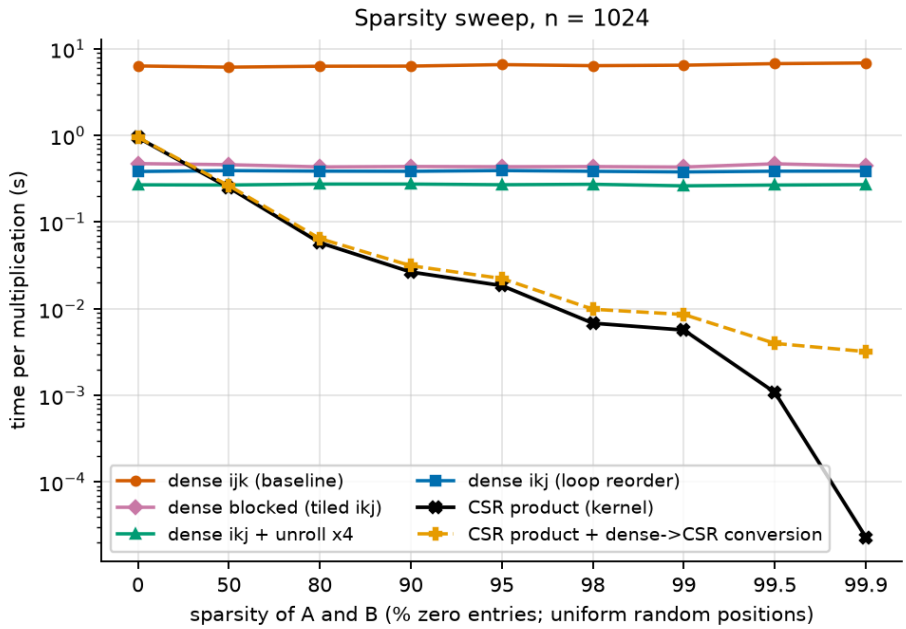


Figure 5. n = 1024: time per multiplication versus sparsity of A and B (uniform positions). The dense methods do not depend on the number of zeros; the CSR kernel does; the dashed line adds the dense→CSR conversion (A and B).

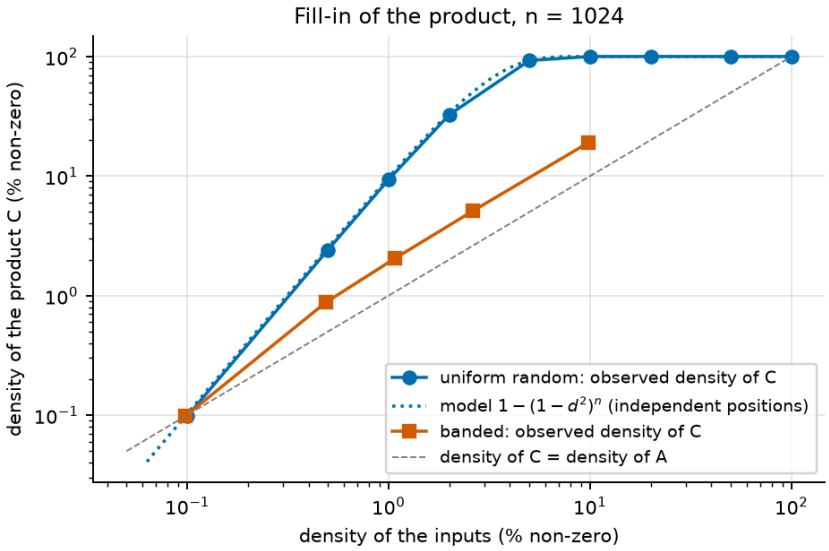


Figure 6. Density of the product versus density of the inputs (n = 1024). Uniform random inputs follow $1-(1-d^2)^n$; banded inputs stay sparse.

Table 7. Size scaling at 99 % zeros (uniform positions). Dense columns are only filled where a dense run was generated and completed (n ≤ 4096; ijk stops at the time budget).

n	nnz(A)	nnz(C)	density of C	CSR product	dense→CSR (A+B)	dense ijk	best dense	best dense / CSR	peak RSS CSR run	peak RSS dense run
256	768	2,279	3.48%	50.1 µs	183.8 µs	44.02 ms	4.53 ms (unroll4)	90.5×	8.3 MiB	9.6 MiB
512	2,560	12,565	4.79%	575.4 µs	704.4 µs	767.35 ms	36.54 ms (unroll4)	63.5×	8.6 MiB	14.2 MiB
1024	10,240	97,951	9.34%	5.54 ms	2.91 ms	6.34 s	266.29 ms (unroll4)	48.0×	11.2 MiB	32.2 MiB
2048	40,960	747,379	17.82%	14.10 ms	11.83 ms	56.15 s	2.74 s (unroll4)	194.0×	22.1 MiB	104.2 MiB
4096	167,936	5,669,258	33.79%	111.44 ms	54.90 ms	–	21.84 s (unroll4)	196.0×	114.8 MiB	392.3 MiB
8192	671,744	37,693,834	56.17%	712.66 ms	–	–	–	–	455.2 MiB	–

Table 8. Size scaling at 99.9 % zeros (uniform positions). Only the blocked dense method was run, up to n = 4096.

n	nnz(A)	nnz(C)	density of C	CSR product	dense→CSR (A+B)	dense ijk	best dense	best dense / CSR	peak RSS CSR run	peak RSS dense run
1024	1,024	1,024	0.10%	18.2 µs	2.60 ms	–	431.97 ms (blocked)	23798.2×	8.4 MiB	32.2 MiB
2048	4,096	8,188	0.20%	206.3 µs	11.84 ms	–	3.52 s (blocked)	17063.3×	8.5 MiB	104.2 MiB
4096	16,384	65,420	0.39%	2.20 ms	44.16 ms	–	28.43 s (blocked)	12926.0×	9.9 MiB	392.2 MiB
8192	65,536	522,511	0.78%	26.52 ms	–	–	–	–	16.4 MiB	–
16384	262,144	4,163,606	1.55%	285.04 ms	–	–	–	–	62.3 MiB	–
32768	1,081,344	35,116,014	3.27%	2.88 s	–	–	–	–	693.8 MiB	–

Table 9. Banded inputs at n = 1024 (|i-j| ≤ w). The product has bandwidth 2w, so it stays sparse.

w	nnz/row	nnz(A)	nnz(C)	density of C	CSR product	dense→CSR	dense ijk	best dense	best dense / CSR
0	1	1,024	1,024	0.10%	19.9 µs	2.71 ms	6.40 s	269.35 ms (unroll4)	13516.0×
2	5	5,114	9,196	0.88%	156.0 µs	2.76 ms	6.80 s	271.32 ms (unroll4)	1739.0×
5	11	11,234	21,394	2.04%	366.7 µs	2.70 ms	6.04 s	266.50 ms (unroll4)	726.7×
13	27	27,466	53,570	5.11%	1.40 ms	2.90 ms	6.36 s	268.12 ms (unroll4)	192.2×
51	103	102,820	199,414	19.02%	11.03 ms	3.02 ms	6.74 s	271.31 ms (unroll4)	24.6×

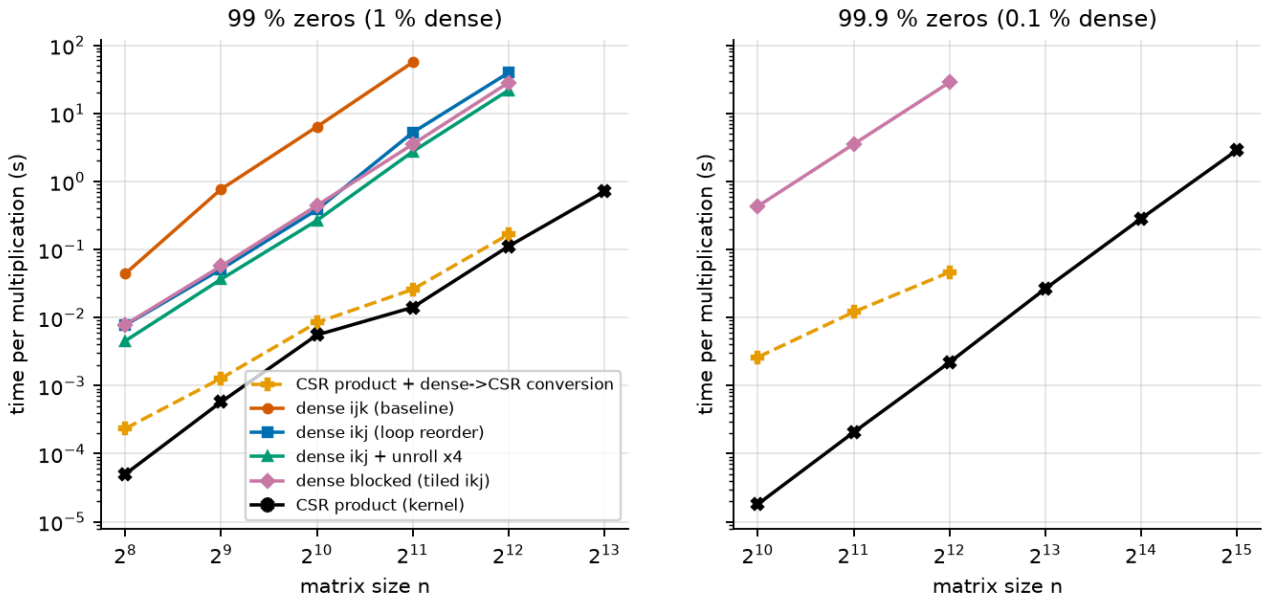


Figure 7. Size scaling at fixed sparsity (uniform positions). The dense methods exist up to n = 4096 (ijk up to 2048); CSR continues to n = 8192 (99 %) and 32768 (99.9 %).

Table 10. Banded inputs with w = 5, size scaling.

n	nnz(A)	nnz(C)	CSR product	dense→CSR	dense blocked	peak RSS CSR run	peak RSS dense run
1024	11,234	21,394	372.5 µs	2.71 ms	441.79 ms	8.7 MiB	32.2 MiB
2048	22,498	42,898	742.2 µs	10.97 ms	3.52 s	9.2 MiB	104.2 MiB
4096	45,026	85,906	1.56 ms	47.37 ms	28.33 s	10.4 MiB	392.2 MiB
8192	90,082	171,922	3.25 ms	–	–	12.4 MiB	–
16384	180,194	343,954	6.04 ms	–	–	16.7 MiB	–
32768	360,418	688,018	12.14 ms	–	–	25.1 MiB	–

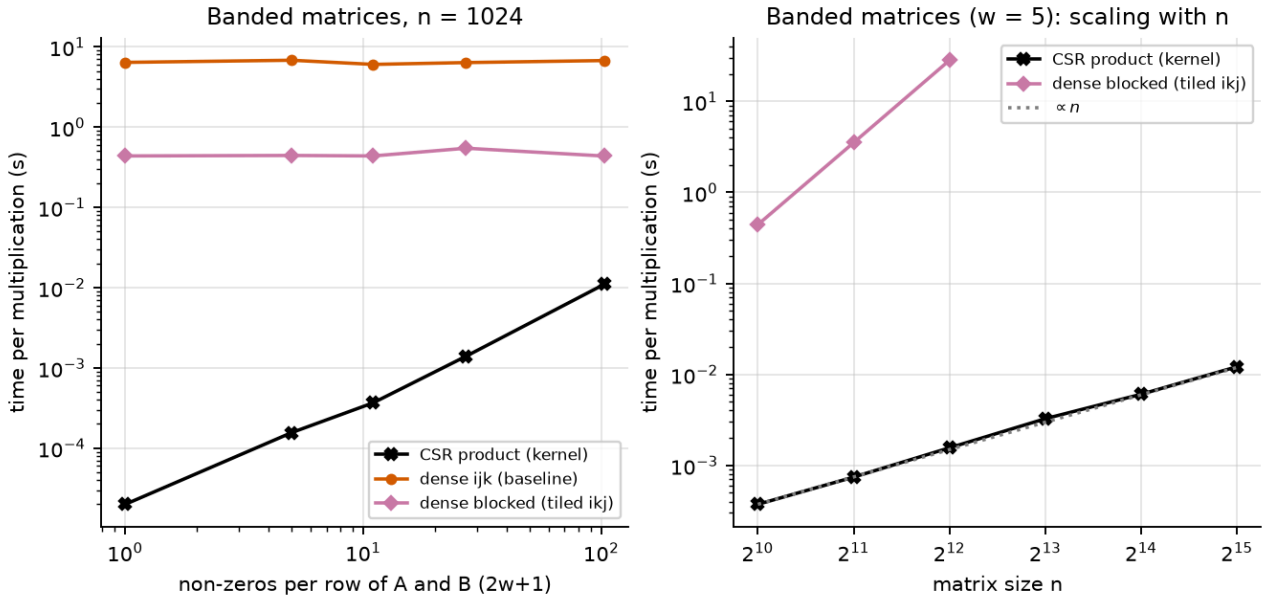


Figure 8. Banded inputs: left, effect of the bandwidth at n = 1024; right, scaling with n at w = 5 (the dotted line is linear).

4.4 Memory and limits

Table 11. Peak working set of the dense benchmark processes (GetProcessMemoryInfo, PeakWorkingSetSize) vs the analytic array size.

n	3 x 8n² B (arrays)	ijk (baseline)	ikj (loop reorder)	ikj + unroll x4	blocked (tiled ikj)
512	6.0 MiB	14.2 MiB	14.2 MiB	14.2 MiB	14.2 MiB
768	13.5 MiB	21.7 MiB	21.7 MiB	21.7 MiB	21.7 MiB
1024	24.0 MiB	32.2 MiB	32.2 MiB	32.2 MiB	32.2 MiB
1536	54.0 MiB	62.2 MiB	62.2 MiB	62.2 MiB	62.2 MiB
2048	96.0 MiB	104.3 MiB	104.2 MiB	104.2 MiB	104.2 MiB
3072	216.0 MiB	–	224.2 MiB	224.2 MiB	224.2 MiB
4096	384.0 MiB	–	392.2 MiB	392.2 MiB	392.3 MiB

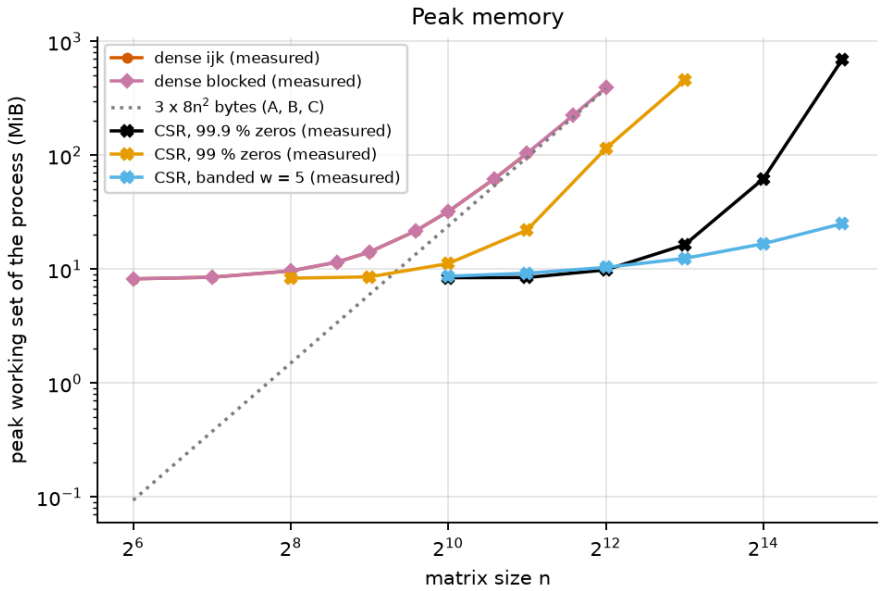


Figure 9. Peak working set of the benchmark processes versus n, dense and CSR runs, with the analytic size of three dense matrices.

Limits. Dense: ijk completed up to n = 2048 (58 s per product) and timed out at 3072; ikj, unroll4 and blocked completed every size generated, up to 4096 (24–41 s), so 4096 is the largest *tested* size, not a limit of the methods (dense inputs were not generated above n = 4096; 8192 would need 1.6 GB for the arrays and about 3 min per product). CSR: largest feasible size 8192 at 99 % zeros (the product is then 56 % dense and the run needs 455 MiB; n = 16384 was not attempted because the product would be almost dense, ≈3 GB of CSR output plus the growth copy, above the 4 GB budget), and 32768 at 99.9 % and for the banded matrices (largest attempted; no failure occurred). Three dense matrices at n = 32768 would need 25.8 GB, more than the 13.8 GB of RAM. Non-OK rows (timeouts and not-attempted entries, with reasons) are listed in `results/non_ok_runs.csv`.

5. Discussion

Part A. *H1 is supported:* loop reordering alone gives 16.2× (n = 1024) and 11.8× (n = 2048), and 1.9–16× for n ≤ 768, raising the throughput from 0.3 to 3.4–5.3 GFLOP/s. The change removes the stride-n access to B; this is the most plausible cause, in line with the literature [5, 8] and with the power-of-two irregularities of ijk, but we measured no cache counters, so it is an interpretation of the timings, not a direct observation. *H2 is supported:* unrolling adds a further 1.47× (n = 1024) to 1.81× (2048) over ikj, so it is the fastest method at every tested n (1.74× at 4096: 23.4 s versus 40.7 s). Per four multiply-adds ikj performs 8 loads and 4 stores of data, the unrolled loop 5 loads and 1 store; the observed gain is consistent with fewer memory operations in a scalar kernel, although we did not isolate this. *H3 is partly supported:* blocking was slower than ikj for n ≤ 1024 (456 versus 404 ms at 1024; the extra loop overhead is not repaid while B, 8 MB at n = 1024, still

fits in the 16 MB L3) and faster once B exceeds it: 1.31× (n = 1536, B = 18 MB), 1.39× (2048) and 1.43× (4096). This coincidence between the cache size and the onset of the benefit supports the capacity explanation. However, blocked did *not* beat unroll4 at any size (3.54 versus 2.72 s at 2048), because our tile kernel is not unrolled. Combining tiling with unrolling or register blocking is the obvious next step and was not tested, so we cannot say which of the two is better in a combined kernel. We did not test Strassen. The gains concern this machine's cache sizes and a scalar-code compiler setting; with vectorised code the balance between loads and arithmetic, and thus the benefit of each technique, would change.

Part B. *H4 is refuted in its numbers, supported in its direction.* The CSR product is faster than the best dense method already at 50 % zeros (1.07×; 1.02× including conversion), not only at 90 %: the crossover lies between 0 % (CSR 3.5× *slower* than unroll4, 0.94 s versus 0.27 s) and 50 % zeros; we did not test values in between. Against the ijk baseline CSR is faster at every tested sparsity, even at 0 % zeros (6.7×), but that is the access-pattern effect of Part A, not a sparsity effect, which is why the comparison with the best dense method is the meaningful one. Above 90 % zeros the advantage grows quickly (10× at 90 %, 46× at 99 %, 248× at 99.5 %). The product of random matrices is fully dense up to 90 % zeros (Figure 6, observed densities match $1-(1-d^2)^n$ closely), and still 33 % dense at 98 %, so the cost of the sparse product is dominated by writing and sorting output entries long before the inputs are sparse enough to make the result sparse.

Conversion. Dense→CSR conversion scans n^2 entries and so costs about 2.9–3 ms at n = 1024 (44 ms at 4096) regardless of sparsity. It is negligible while the kernel takes tens of milliseconds, but at 99.9 % zeros the kernel needs 23 µs and the conversion is 99 % of the total: the speed-up over the best dense method falls from 11,730× (kernel only) to 85× (with conversion). Sparse multiplication therefore pays at very high sparsity only if the data are kept in sparse form (as in the CSR-only scaling runs). *Structure.* At the same density the input pattern matters (H5 supported): 1 % uniform (10 non-zeros per row) takes 5.5 ms, banded w = 5 (11 per row, density 1.07 %) 0.37 ms, 15× less, because the uniform product fills in to 9.3 % (98,000 entries) against 2.0 % (21,000) and its accesses are scattered, while banded rows are contiguous. The same counts explain scaling: banded CSR time grows linearly with n (slope 1.01; 12 ms at n = 32768), uniform 99 % with slope 2.66 (256–8192) and uniform 99.9 % with slope 3.46 (1024–32768; the number of non-zeros per row grows proportionally to n, so the work is again of order n^3, with a much smaller constant than the dense one). At 99.9 % and n = 32768 the sparse product takes 2.9 s and 694 MiB; the dense matrices would not fit in memory.

Memory. The dense processes use the analytic $3\times8n^2$ bytes plus a fixed ≈8 MiB (runtime and I/O buffers; the n = 64 process peaks at 8.2 MiB) and do not depend on the method or the sparsity. CSR processes need memory in proportion to the non-zeros of A, B and the product: 8–11 MiB for n = 1024 at 99–99.9 % zeros against 32 MiB dense, and the output dominates when the product fills in (455 MiB at n = 8192, 99 %, 37.7 M output entries × 12 B). Peak working set is a process-level metric and is not the same as the live matrix size.

Limitations. (i) One laptop, one compiler setting (`-O2`, scalar code), one run order; between-session drift of about 30 % was observed, so the tile sweep and D1 are not directly comparable. (ii) Tiles were tuned on the evaluation machine. (iii) Only 3 repetitions at n > 2048 because of the run time. (iv) The sparse and dense timing boundaries differ slightly (output allocation is inside the sparse timer). (v) Dense inputs above n = 4096 were not generated, so the sparse/dense comparison is limited to n ≤ 4096, and dense runs for the 99.9 % and banded size sweeps use the blocked kernel instead of the fastest dense kernel (unroll4); this understates the dense speed by about 20 % there. (vi) Cache and TLB explanations are hypotheses consistent with the data, not counter measurements. (vii) The crossover between 0 and 50 % zeros was not located more finely; the product's sorting step and the int32 indices are design choices of our CSR kernel, and a library implementation (e.g. SciPy or MKL) could behave differently. (viii) Results for uniform and banded patterns do not cover other structures (e.g. power-law row lengths, block sparsity).

6. Conclusions

1. Within the tested scope (Ryzen 7 7730U, gcc 16.1 `-O2`, scalar single-thread code, binary64, kernel-only timing), three optimised dense kernels are correct and bit-identical to the baseline.

2. Loop reordering gives 8.5–16× for n ≥ 768; unrolling by 4 adds 1.5–1.8× (best overall: up to 24× over ijk); blocking helps ikj by 1.3–1.4× only when B exceeds the L3 cache, and was not better than the unrolled kernel.
3. CSR multiplication beat the best dense method from 50 % zeros at n = 1024 (not tested between 0 and 50 %), by 10× at 90 % and 46× at 99 %; conversion and output fill-in reduce the advantage, and the input structure (random versus banded) changes the cost by more than an order of magnitude at equal density.
4. Sparse storage extends the feasible sizes: 32768 at 99.9 % zeros in 694 MiB where dense is infeasible in RAM. The largest tested feasible dense size was 4096 (2048 for ijk).

Unresolved: the best combination of tiling and unrolling, the exact sparsity crossover, and hardware-level confirmation of the cache explanations.

AI-use disclosure

In line with the course policy, the AI assistant Claude (Anthropic) was used to plan the experimental design, draft and debug the C kernels, the benchmark and analysis scripts and the tests, and to draft and revise the English of this report. All numbers in the report were produced by running the code in the repository (tables and figures are generated by `scripts/analyze.py`); no result was generated by the AI tool. The author is responsible for understanding the code, checking the measurements and references, and defending the conclusions.

Reproducibility materials

Folder `Assignment02_AlejandroSantana_Materials` (and the repository https://github.com/aimarsantana/BigData-Assignment02-MatMul): `src/c/matmul2.c`, `README.md` with exact build and run commands, `config/benchmark_config.json`, input generators and manifest (`data/inputs/manifest.json`), raw samples and run logs (`results/`), validation output, environment description, and the scripts that produce all tables and figures. `run_all.ps1` rebuilds, validates, measures, analyses and regenerates this report. Reused from Assignment 1: the ijk kernel and the dense input generator (same seeds and hashes).

References

1. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.
2. M. S. Lam, E. E. Rothberg, and M. E. Wolf, “The cache performance and optimizations of blocked algorithms,” in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*, 1991, pp. 63–74.
3. F. G. Gustavson, “Two fast algorithms for sparse matrices: Multiplication and permuted transposition,” *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250–269, 1978.
4. Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.
5. U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Tech. Rep., Nov. 2007.
6. K. Goto and R. A. van de Geijn, “Anatomy of high-performance matrix multiplication,” *ACM Trans. Math. Softw.*, vol. 34, no. 3, Art. 12, 2008.
7. V. Strassen, “Gaussian elimination is not optimal,” *Numer. Math.*, vol. 13, no. 4, pp. 354–356, 1969.
8. J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
9. N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.
10. Microsoft, “PROCESS_MEMORY_COUNTERS structure (psapi.h)” and “GetProcessMemoryInfo function,” Microsoft Learn, Windows API documentation.
11. T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2006.